>_ MCMP113 · Week 13

Value-Returning and void Functions · Scope

The return statement, bool predicates, returning arrays and tuples, pure functions, recursion, block scope and lifetime, local functions

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 13 slides and add what does not fit on a slide: the reasoning behind each rule, call-stack traces, scope diagrams, the twelve example programs (complete, or as an excerpt for the longer Examples 10–12) with real sample runs, the real compiler messages for the typical mistakes, a glossary and graded exercises. Every listing is read from the tested program in the `code` folder (`13_01` … `13_12`); open it in VS Code and run it while you read. Typed input in the sample runs is shown in **amber**. As in Week 12, methods are *local functions* written at the bottom of `Program.cs`, each with a one-line comment. Recursion traces and "spot the scope error" questions are typical final-exam questions.

Contents

1 Learning outcomes 2
2 The return statement 2
3 bool predicates and the T.C. Kimlik check 5
4 Returning arrays and several values 7
5 Composition, pure functions and side effects 10
6 Recursion 12
7 Scope and lifetime 15
8 Testing methods and small utility libraries 18
9 Common mistakes 21
10 Summary 21
11 Glossary (English – Turkish) 22
12 Exercises 22
13 References 24

>_ 1 Learning outcomes

After this week you should be able to:

1. explain what `return` does (hand back a value *and* end the method), use early returns and guard clauses, and recognise the compiler error CS0161;
2. choose between a `void` method (an action) and a value-returning method (a computation), and prefer returning a result over printing it inside;
3. write `bool` predicates such as `IsEven`, `IsPrime` and `IsValidTCKimlik`;
4. return an array from a method, and return two or more values with `out` parameters or a tuple;
5. compose method calls, and tell a pure function from a method with side effects;
6. write simple recursive methods (factorial, sum of digits, GCD, Fibonacci), name the base case and trace the call stack;
7. determine the scope and the lifetime of a variable, explain CS0103, CS0136, CS0128 and CS8421, and prefer `static` local functions;
8. test methods automatically with known inputs and PASS/FAIL output.

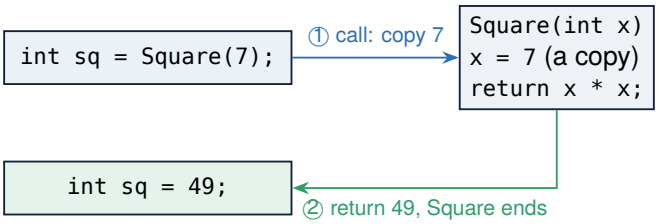
>_ 2 The return statement

2.1 Value-returning methods

A method with a return type other than `void` *computes* something and gives it back to the caller with `return`. Two things happen when a `return` statement runs:

1. the expression after `return` is evaluated and becomes the **return value**;
2. the method *ends immediately*: no statement after the `return` runs, and execution continues in the caller.

In the caller, the whole call expression is replaced by the returned value. After `int sq = Square(7);` the line has effectively become `int sq = 49;`. Because a call to a value-returning method *is* a value, it may appear wherever a value of that type may appear: in an arithmetic expression (`Square(3) + Square(4)`), in a condition, in an interpolated string, or as an argument of another call (Section 5).



2.2 void methods

A `void` method performs an *action* — printing, changing the elements of an array, later writing a file — and gives nothing back. Its call is a statement on its own (`PrintTitle("Early return");`); using it as a value is an error (CS0029, Table 1). A `void` method may contain `return;` without a value to leave early.

	void method	value-returning method
Purpose	an action (print, change data)	a computation (a result)
Header	`static void PrintTitle(string title)`	`static int Square(int x)`
Call	a statement: `PrintTitle("A");`	inside an expression: `int s = Square(7);`
`return`	optional, without a value: `return;`	required on every path, with a value

Design rule: return, do not print

A method that computes something should *return* the result and leave printing to the caller. `Average(a)` can be printed, compared with 60, stored in an array or checked by a test; a `PrintAverage(a)` that writes the number to the console can only print it. Printing belongs to `void` methods whose *job* is output, such as `PrintTitle` or `PrintArray`.

2.3 Early return and guard clauses

Because `return` ends the method, a method may contain several `return` statements; for each call exactly one of them runs. A **guard clause** handles the invalid or special case at the very beginning and returns at once, so that the rest of the method can rely on valid data. In Example 1, `LetterGrade` first rejects scores outside 0 . . . 100; the switch expression (Week 5) then only sees valid scores. `PrintQuotient` is a `void` method that leaves with `return;` when the divisor is zero.

2.4 Example 1 – value-returning and void methods

Listing 1: 13_01_ReturnBasics/Program.cs (without `PrintTitle` and `Square`, lines 21–35)

```
1  // MCMP113 · Week 13 · Example 1 — return: value-returning and void methods
2  // A value-returning method hands a result back; return also ENDS the method at once.
3
4  PrintTitle("Value-returning");
5  int sq = Square(7);                          // the call is replaced by its value
6  Console.WriteLine($"Square(7) = {sq}");
7  Console.WriteLine($"Square(3) + Square(4) = {Square(3) + Square(4)}");
8  Console.WriteLine($"Sign(-8) = {Sign(-8)}, Sign(0) = {Sign(0)}, Sign(5) = {Sign(5)}");
9
10 PrintTitle("Early return");
11 int[] scores = { 95, 87, 72, 64, 41, 105 };
12 foreach (int score in scores)
13 {
14     Console.WriteLine($"{score,4} -> {LetterGrade(score)}");
15 }
16
17 PrintTitle("void with return;");
18 PrintQuotient(17, 5);
19 PrintQuotient(17, 0);
```

... lines 21–35 omitted ...

```
36 // Returns 1, -1 or 0; every path ends with a return
37 static int Sign(int n)
38 {
39     if (n > 0)
40     {
41         return 1;
42     }
43     if (n < 0)
44     {
45         return -1;
46     }
```

```
    return 0;
}

// Returns the letter grade; the first return that runs ends the method
static string LetterGrade(int score)
{
    if (score < 0 || score > 100)
    {
        return "invalid";                   // guard clause: bad input first
    }
    return score switch                     // example scale (see Week 5)
    {
        >= 90 => "AA",
        >= 85 => "BA",
        >= 80 => "BB",
        >= 75 => "CB",
        >= 70 => "CC",
        >= 65 => "DC",
        >= 60 => "DD",
        _ => "FF"
    };
}

// Prints a / b, or a message; "return;" leaves a void method early
static void PrintQuotient(int a, int b)
{
    if (b == 0)
    {
        Console.WriteLine($"{a} / {b}: cannot divide by zero");
        return;                             // nothing after this line runs
    }
    Console.WriteLine($"{a} / {b} = {a / b} remainder {a % b}");
}
```

Sample run (lines 1–9)

```
==================
  Value-returning
==================
Square(7) = 49
Square(3) + Square(4) = 25
Sign(-8) = -1, Sign(0) = 0, Sign(5) = 1
==============
  Early return
==============
```

Sample run (lines 10–20)

```
  95 -> AA
  87 -> BA
  72 -> CC
  64 -> DD
  41 -> FF
 105 -> invalid
====================
  void with return;
====================
17 / 5 = 3 remainder 2
17 / 0: cannot divide by zero
```

2.5 Every path must return

For a value-returning method the compiler checks *every* way through the method. If some path reaches the closing brace without a `return`, the program does not compile. The compiler does not look at the values: in the version of `Sign` below the case `n == 0` has no `return`, and even if you "know" that n is never 0, the error remains.

```
static int Sign(int n)
{
    if (n > 0)
    {
        return 1;
    }
    if (n < 0)
    {
        return -1;
    }
}   // n == 0: no return!
```

```
dotnet build

error CS0161: 'Sign(int)': not all code paths return a value
```

The fix is the last line of Example 1's `Sign`: `return 0;`. Table 1 lists the other `return` mistakes; each message was reproduced with `dotnet build`.

Table 1: Compiler errors around `return` (real messages, .NET 10 SDK).

Code	Situation	Message
CS0161	a path without `return`	`'Sign(int)': not all code paths return a value`
CS0127	`return 0;` in a `void` method	`Since 'PrintLine()' returns void, a return keyword must not be followed by an object expression`
CS0126	`return;` in an `int` method	`An object of a type convertible to 'int' is required`
CS0029	`int x = PrintTitle("A");`	`Cannot implicitly convert type 'void' to 'int'`
CS0266	`return n / 2.0;` in an `int` method	`Cannot implicitly convert type 'double' to 'int'. An explicit conversion exists (are you missing a cast?)`

>_ 3 bool predicates and the T.C. Kimlik check

A **predicate** is a method that returns `bool`: it answers a yes/no question. Name it like a question — `IsEven`, `IsPrime`, `HasDigit`, `CanVote` — so that an `if` reads like English: `if (IsPrime(n))`. Two style rules:

- Return the condition itself: `return n % 2 == 0;` instead of `if (n % 2 == 0) { return true; } return false;`.
- Do not compare a `bool` with `true`: write `if (IsEven(n))`, not `if (IsEven(n) == true)`.

A predicate may also stop as soon as the answer is known: `IsAllDigits` returns `false` at the first character that is not a digit, and `true` only after the whole loop.

Trap: odd numbers and negative values

`n % 2 == 1` is *not* a correct test for "odd": in C# the remainder has the sign of the dividend, so `-7 % 2` is -1. Use `n % 2 != 0`, or `!IsEven(n)`.

3.1 The Week 1 algorithm as a method

In Week 1 we described the check digits of the 11-digit T.C. Kimlik No $d_1 d_2 \dots d_{11}$:

$$d_1 \neq 0, \qquad d_{10} = \big(7(d_1 + d_3 + d_5 + d_7 + d_9) - (d_2 + d_4 + d_6 + d_8)\big) \bmod 10,$$
$$d_{11} = (d_1 + \cdots + d_{10}) \bmod 10.$$

In Week 4 we met the problem that $7 \cdot \text{odd} - \text{even}$ can be negative while the C# operator `%` keeps the sign. Now the whole check becomes one reusable predicate `IsValidTCKimlik(string id)`, with a small helper `Mod` for the mathematical remainder $((a \,\%\, m) + m) \,\%\, m \in \{0, \ldots, m-1\}$.

IsValidTCKimlik(id)

IF length $\neq 11$ **OR** not all digits **OR** $d_1 = 0$ **THEN**
  RETURN false
END IF
$odd \leftarrow d_1 + d_3 + d_5 + d_7 + d_9$
$even \leftarrow d_2 + d_4 + d_6 + d_8$
$c_{10} \leftarrow (7 \cdot odd - even) \bmod 10$
$c_{11} \leftarrow (odd + even + c_{10}) \bmod 10$
RETURN $d_{10} = c_{10}$ **AND** $d_{11} = c_{11}$

Trace for the synthetic number `19090909018`:

step	value
$odd = 1+0+0+0+0$	1
$even = 9+9+9+9$	36
$7 \cdot odd - even$	-29
`-29 % 10` (C#)	-9
`Mod(-29, 10)`	$1 = d_{10}$
$(1+36+1) \bmod 10$	$8 = d_{11}$

With the plain `%` the computed d_{10} would be -9, which never equals a digit, and a correct number would be rejected.

Synthetic numbers only

All numbers in these notes (`12345678950`, `19090909018`, ...) are invented. Never type, store, test with or share a real person's T.C. Kimlik No — it is personal data protected by law (KVKK).

3.2 Example 2 – IsEven, IsAllDigits, IsValidTCKimlik

Listing 2: 13_02_TCKimlik/Program.cs (without `IsEven`, lines 26–31: `return n % 2 == 0;`)

```
1  // MCMP113 · Week 13 · Example 2 – bool predicates: IsEven, IsAllDigits, IsValidTCKimlik
2  // The Week 1 check-digit algorithm as a method. Only SYNTHETIC numbers are used.
3
4  Console.WriteLine($"IsEven(14) = {IsEven(14)}, IsEven(-7) = {IsEven(-7)}");
5  Console.WriteLine($"-29 % 10 = {-29 % 10}, but Mod(-29, 10) = {Mod(-29, 10)}");
6  Console.WriteLine();
7
8  string[] tests = { "12345678950", "19090909018", "12345678951", "12345678590",
9                     "02345678950", "1234567895", "1234567895A" };
10 foreach (string t in tests)
11 {
12     Console.WriteLine($"{t,-12} -> {IsValidTCKimlik(t)}");
13 }
14
15 Console.Write("Number to check: ");
16 string input = Console.ReadLine();
17 if (IsValidTCKimlik(input))
18 {
19     Console.WriteLine("The check digits are correct.");
20 }
21 else
22 {
23     Console.WriteLine("Not a valid number (typing error?).");
24 }
```

... lines 26–31 omitted ...

```
32 // Mathematical remainder 0..m-1, also for negative a (C# % keeps the sign of a)
33 static int Mod(int a, int m)
34 {
```

```
    return ((a % m) + m) % m;
}

// True if s is not empty and every character is a digit 0-9
static bool IsAllDigits(string s)
{
    if (s.Length == 0)
    {
        return false;
    }
    foreach (char c in s)
    {
        if (c < '0' || c > '9')
        {
            return false;                    // one bad character is enough
        }
    }
    return true;
}

// True if id has 11 digits, d1 != 0 and both check digits d10, d11 are right
static bool IsValidTCKimlik(string id)
{
    if (id.Length != 11 || !IsAllDigits(id) || id[0] == '0')
    {
        return false;
    }
    int[] d = new int[11];
    for (int i = 0; i < 11; i++)
    {
        d[i] = id[i] - '0';                  // char '7' -> int 7
    }
    int oddSum = d[0] + d[2] + d[4] + d[6] + d[8];
    int evenSum = d[1] + d[3] + d[5] + d[7];
    int d10 = Mod(7 * oddSum - evenSum, 10); // may be negative before Mod!
    int d11 = (oddSum + evenSum + d10) % 10;
    return d[9] == d10 && d[10] == d11;
}
```

Sample run 1

```
IsEven(14) = True, IsEven(-7) = False
-29 % 10 = -9, but Mod(-29, 10) = 1

12345678950  -> True
19090909018  -> True
12345678951  -> False
12345678590  -> False
02345678950  -> False
1234567895   -> False
1234567895A  -> False
Number to check: 12345678950
The check digits are correct.
```

Sample run 2 (last lines)

```
Number to check: 12345678905
Not a valid number (typing error?).
```

Each test number has one reason: valid; valid with a negative $7 \cdot odd - even$; wrong d_{11}; two neighbouring digits swapped; leading zero; only ten digits; a letter. A single wrong digit is always detected.

>_ 4 Returning arrays and several values

4.1 Returning an array

A method may create an array, fill it and return it. The local variable that holds the array (`result` in `ReadArray`) disappears when the method ends, but the array itself lives on: the caller now holds

a reference to it (arrays are reference types, Week 9). Methods like `ReverseCopy` and `EvensOnly` build a *new* array and leave the original untouched. `EvensOnly` needs two passes because an array cannot grow: pass 1 counts the even values, pass 2 creates an array of exactly that length and copies them.

4.2 Example 3 – ReadArray, ReverseCopy, EvensOnly

Listing 3: 13_03_ReturnArrays/Program.cs (without `ReverseCopy` and `PrintArray`)

```
1  // MCMP113 · Week 13 · Example 3 — methods that return arrays
2  // ReadArray creates and fills a new array; ReverseCopy and EvensOnly leave the original unchanged.
3
4  Console.Write("How many numbers? ");
5  int n = int.Parse(Console.ReadLine());
6  int[] data = ReadArray(n);
7
8  int[] backwards = ReverseCopy(data);
9  int[] evens = EvensOnly(data);
10
11 PrintArray("data    ", data);
12 PrintArray("reversed", backwards);
13 PrintArray("evens   ", evens);
14 PrintArray("data    ", data);          // still in the original order
15
16 // Reads n integers from the keyboard and returns them in a new array
17 static int[] ReadArray(int n)
18 {
19     int[] result = new int[n];
20     for (int i = 0; i < n; i++)
21     {
22         Console.Write($"  value {i + 1}: ");
23         result[i] = int.Parse(Console.ReadLine());
24     }
25     return result;                      // the caller receives the reference
26 }
```

... lines 28–38 omitted ...

```
39 // Returns a new array that holds only the even elements of a (two passes)
40 static int[] EvensOnly(int[] a)
41 {
42     int count = 0;
43     foreach (int x in a)                // pass 1: how big must the result be?
44     {
45         if (x % 2 == 0)
46         {
47             count++;
48         }
49     }
50     int[] result = new int[count];
51     int k = 0;
52     foreach (int x in a)                // pass 2: copy the even values
53     {
54         if (x % 2 == 0)
55         {
56             result[k] = x;
57             k++;
58         }
59     }
60     return result;
61 }
```

`ReverseCopy` (lines 28–38) creates `int[] copy = new int[a.Length];`, writes `copy[a.Length - 1 - i] = a[i];` in a loop and returns `copy`.

```
Sample run

How many numbers? 5
  value 1: 8
  value 2: 3
  value 3: 12
  value 4: 7
  value 5: 6
data    : [8, 3, 12, 7, 6]  (Length 5)
reversed: [6, 7, 12, 3, 8]  (Length 5)
evens   : [8, 12, 6]  (Length 3)
data    : [8, 3, 12, 7, 6]  (Length 5)
```

4.3 Two results: out parameters or a tuple

`return` gives back exactly *one* value. When a method has two results (the minimum *and* the maximum, the quotient *and* the remainder) there are two ways:

- **out parameters**: the method writes into variables of the caller; every `out` parameter must be assigned before the method ends. You already know the pattern from `int.TryParse(text, out int value)`.
- a **tuple**: `(int min, int max)` is *one* value with two named parts. The caller can deconstruct it, `(int min, int max) = MinMax(temps);`, or keep it whole and write `result.min`.

For new code tuples are usually clearer; `out` remains common in the .NET library (`TryParse`).

4.4 Example 4 – out parameters and tuples

Listing 4: 13_04_MultipleResults/Program.cs (without `MinMaxOut`, lines 22–39)

```
1  // MCMP113 · Week 13 · Example 4 — returning two values: out parameters and tuples
2  // Same job (min and max of an array) written twice; plus a quotient/remainder tuple.
3
4  int[] temps = { 14, 9, 21, 17, 6, 12 };
5  Console.WriteLine($"temps: {string.Join(" ", temps)}");
6
7  // Version 1: out parameters (like int.TryParse)
8  MinMaxOut(temps, out int low, out int high);
9  Console.WriteLine($"out:   min = {low}, max = {high}");
10
11 // Version 2: a tuple, deconstructed into two new variables
12 (int min, int max) = MinMax(temps);
13 Console.WriteLine($"tuple: min = {min}, max = {max}, range = {max - min}");
14
15 // A tuple can also be kept whole; the parts have names
16 var result = MinMax(new int[] { 3, -5, 8 });
17 Console.WriteLine($"whole: result.min = {result.min}, result.max = {result.max}");
18
19 (int q, int r) = DivMod(47, 6);
20 Console.WriteLine($"47 = 6 * {q} + {r}");
```

... lines 22–39 omitted ...

```
40 // Returns the smallest and the largest element of a as a named tuple
41 static (int min, int max) MinMax(int[] a)
42 {
43     int min = a[0];
44     int max = a[0];
45     foreach (int x in a)
46     {
47         if (x < min)
48         {
```

```
            min = x;
        }
        if (x > max)
        {
            max = x;
        }
    }
    return (min, max);                  // two values in one return
}

// Returns the integer quotient and the remainder of a / b
static (int quotient, int remainder) DivMod(int a, int b)
{
    return (a / b, a % b);
}
```

`MinMaxOut` has the same loop; its header is `static void MinMaxOut(int[] a, out int min, out int max)` and its first statements `min = a[0]; max = a[0];` assign both `out` parameters, as required.

Sample run

```
temps: 14 9 21 17 6 12
out:   min = 6, max = 21
tuple: min = 6, max = 21, range = 15
whole: result.min = -5, result.max = 8
47 = 6 * 7 + 5
```

>_ 5 Composition, pure functions and side effects

5.1 Composition

An argument may be any expression of the right type, including another method call. C# evaluates the inner call first and passes its value to the outer one:

$$\texttt{Max(Max(8, 15), 11)} \;\to\; \texttt{Max(15, 11)} \;\to\; 15,$$
$$\texttt{WithVat(Discount(250, 10))} \;\to\; \texttt{WithVat(225)} \;\to\; 270.$$

With a two-parameter `Max` we obtain the maximum of three values; with two small price methods we build a pipeline ($250 \cdot 0.9 = 225$, $225 \cdot 1.20 = 270$).

5.2 Pure functions

A method is **pure** if (1) its result depends only on its arguments and (2) it changes nothing outside itself: it does not print, does not modify an argument array and does not touch variables outside the method. `Square`, `Max`, `Average` are pure: called twice with the same input, they give the same output. A method that does anything else has a **side effect**. Side effects are sometimes the purpose of the method (printing, sorting an array in place), but they should be visible in the name and the comment. `CurveAndAverage` in Example 5 is a warning: it silently adds 5 points to the caller's grades, so calling it twice gives two different averages and a grade of 101.

5.3 Example 5 – composition, pure functions, side effects

Listing 5: 13_05_PureFunctions/Program.cs (without `Discount`, lines 27–32)

```
// MCMP113 · Week 13 · Example 5 — composition, pure functions and side effects
// Calls inside calls; a pure function always gives the same result and changes nothing else.

```

```
4  const double VatRate = 0.20;           // a constant may be used everywhere
5  int[] grades = { 72, 58, 91 };
6
7  // Composition: the result of one call is the argument of the next
8  Console.WriteLine($"Max(Max(8, 15), 11) = {Max(Max(8, 15), 11)}");
9  double price = 250;
10 Console.WriteLine($"WithVat(Discount(250, 10)) = {WithVat(Discount(price, 10)):F2} TL");
11
12 // A pure function: same input -> same output, every time
13 Console.WriteLine($"Average: {Average(grades):F2}, again: {Average(grades):F2}");
14 Console.WriteLine($"Passed? {Average(grades) >= 60}");   // the value can be reused
15
16 // A method with a side effect: it changes the caller's array
17 Console.WriteLine($"Average after Curve: {CurveAndAverage(grades):F2}");
18 Console.WriteLine($"Average after Curve: {CurveAndAverage(grades):F2}");
19 Console.WriteLine($"grades is now: {string.Join(" ", grades)}");
20
21 // Returns the larger of a and b
22 static int Max(int a, int b)
23 {
24     return a > b ? a : b;
25 }
```

... lines 27–32 omitted ...

```
33 // Returns the price with VAT added (VatRate is a constant, not a variable)
34 static double WithVat(double price)
35 {
36     return price * (1 + VatRate);
37 }
38
39 // Returns the mean of the array; reads it, changes nothing (pure)
40 static double Average(int[] a)
41 {
42     int sum = 0;
43     foreach (int x in a)
44     {
45         sum += x;
46     }
47     return (double)sum / a.Length;
48 }
49
50 // Adds 5 points to every grade (side effect!) and returns the new mean
51 static double CurveAndAverage(int[] a)
52 {
53     for (int i = 0; i < a.Length; i++)
54     {
55         a[i] += 5;                     // writes into the caller's array
56     }
57     return Average(a);
58 }
```

Sample run

```
Max(Max(8, 15), 11) = 15
WithVat(Discount(250, 10)) = 270.00 TL
Average: 73.67, again: 73.67
Passed? True
Average after Curve: 78.67
Average after Curve: 83.67
grades is now: 82 68 101
```

The constant `VatRate` is declared in the top-level code and used by the `static` method `WithVat`. This is allowed because a `const` never changes (Section 7.4).

>_ 6 Recursion

6.1 A method that calls itself

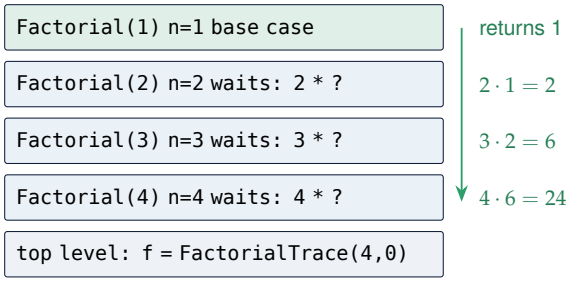
A method is **recursive** if it calls itself. Recursion is not a new statement: it is an ordinary call, and everything from Week 12 still holds (each call gets its own copies of the parameters, the caller waits for the return value). A correct recursive method has two parts:

- a **base case** that answers directly, without a further call;
- a **recursive step** that calls the method with a *smaller* problem, so that the base case is eventually reached.

Factorial is defined recursively in mathematics: $0! = 1! = 1$ and $n! = n \cdot (n-1)!$. The sum of the digits of n is its last digit n mod 10 plus the digit sum of $\lfloor n/10 \rfloor$. Euclid's algorithm from Week 1 is recursive as well: $\gcd(a, b) = \gcd(b, a \bmod b)$ and $\gcd(a, 0) = a$.

6.2 The call stack

Every call gets a **stack frame** — a small memory area for its parameters and local variables — placed on top of the **call stack**. While `Factorial(4)` waits for `Factorial(3)`, both frames exist, each with its *own* variable `n`. The frames pile up ("winding") until the base case returns; then each frame completes its multiplication and is removed ("unwinding"):



FactorialTrace(4) – real output

```
call Factorial(4)
  call Factorial(3)
    call Factorial(2)
      call Factorial(1)
      return 1
    return 2
  return 6
return 24
```

6.3 Example 6 – factorial, sum of digits, GCD and a traced factorial

Listing 6: 13_06_RecursionBasics/Program.cs (lines 1–41; `FactorialTrace` follows in lines 42–58)

```
1  // MCMP113 · Week 13 · Example 6 — recursion: a method that calls itself
2  // Every recursive method needs a BASE CASE that stops, and a step that gets closer to it.
3
4  Console.WriteLine($"Factorial(5) = {Factorial(5)}");
5  Console.WriteLine($"SumOfDigits(2026) = {SumOfDigits(2026)}");
6  Console.WriteLine($"Gcd(84, 36) = {Gcd(84, 36)}");
7  Console.WriteLine();
8  Console.WriteLine("Call stack of FactorialTrace(4):");
9  long f = FactorialTrace(4, 0);
10 Console.WriteLine($"Result: {f}");
11
12 // n! = n * (n-1)!, with 0! = 1! = 1
13 static long Factorial(int n)
14 {
15     if (n <= 1)
16     {
17         return 1;                          // base case
18     }
19     return n * Factorial(n - 1);           // recursive case
20 }
```

```
21
22  // Sum of the decimal digits of n >= 0: last digit + sum of the others
23  static int SumOfDigits(int n)
24  {
25      if (n < 10)
26      {
27          return n;
28      }
29      return n % 10 + SumOfDigits(n / 10);
30  }
31
32  // Euclid (Week 1): gcd(a, b) = gcd(b, a mod b), and gcd(a, 0) = a
33  static int Gcd(int a, int b)
34  {
35      if (b == 0)
36      {
37          return a;
38      }
39      return Gcd(b, a % b);
40  }
```

`FactorialTrace(n, depth)` is `Factorial` with two extra `WriteLines`: `call Factorial(n)` on entry and `return result` before leaving, indented by `2 * depth` spaces; each recursive call passes `depth + 1`.

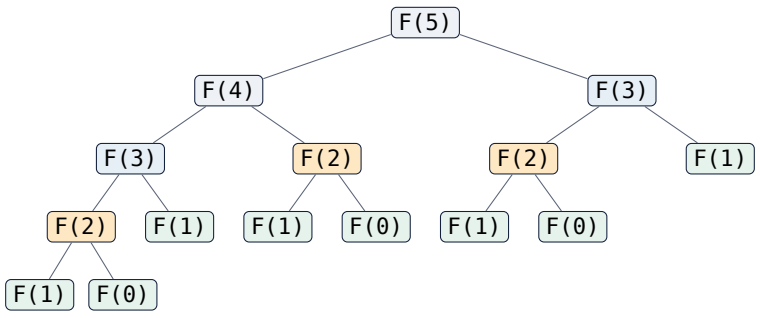
Sample run (first lines; the trace of FactorialTrace(4) is shown above)

```
Factorial(5) = 120
SumOfDigits(2026) = 10
Gcd(84, 36) = 12
```

Traces of the other two methods: `SumOfDigits(2026)` $= 6 +$ `SumOfDigits(202)` $= 6 + 2 +$ `SumOfDigits(20)` $= 6 + 2 + 0 +$ `SumOfDigits(2)` $= 10$, and `Gcd(84, 36)` $\to$ `Gcd(36, 12)` $\to$ `Gcd(12, 0)` $\to 12$ (the Week 1 homework answer).

6.4 Fibonacci: naive recursion versus a loop

The Fibonacci numbers satisfy $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. The recursive method is a direct copy of the definition, but every call makes *two* further calls, and the same values are computed again and again. The call tree for $n = 5$ has 15 nodes; F_3 is computed twice and F_2 three times:



In general the naive method makes $2F_{n+1} - 1$ calls (for $n = 5$: $2 \cdot 8 - 1 = 15$), which grows by a factor of about 1.618 per step. The loop version needs only $n - 1$ passes and two variables. Example 7 counts the recursive calls with a `ref` counter shared by all calls.

6.5 Example 7 – Fibonacci race

Listing 7: 13_07_FibonacciRace/Program.cs

```
// MCMP113 · Week 13 · Example 7 — Fibonacci: naive recursion vs a loop
// Both give the same numbers, but the recursive version repeats work exponentially.

Console.Write("First 10 terms:");
for (int i = 0; i < 10; i++)
{
    Console.Write($" {FibLoop(i)}");
}
Console.WriteLine();

Console.WriteLine("  n        Fib(n)   recursive calls   loop passes");
int[] sizes = { 5, 10, 20, 30, 35 };
foreach (int n in sizes)
{
    long calls = 0;
    long viaRecursion = FibRec(n, ref calls);
    long viaLoop = FibLoop(n);
    string same = viaRecursion == viaLoop ? "" : "  MISMATCH";
    Console.WriteLine($"{n,3} {viaLoop,13} {calls,17} {n - 1,13}{same}");
}

// Fib(0) = 0, Fib(1) = 1, Fib(n) = Fib(n-1) + Fib(n-2); counts every call in 'calls'
static long FibRec(int n, ref long calls)
{
    calls++;
    if (n < 2)
    {
        return n;                           // two base cases: 0 and 1
    }
    return FibRec(n - 1, ref calls) + FibRec(n - 2, ref calls);
}

// The same numbers with a loop: n - 1 passes, two variables
static long FibLoop(int n)
{
    long previous = 0;
    long current = 1;
    if (n == 0)
    {
        return 0;
    }
    for (int i = 2; i <= n; i++)
    {
        long next = previous + current;
        previous = current;
        current = next;
    }
    return current;
}
```

Sample run

```
First 10 terms: 0 1 1 2 3 5 8 13 21 34
  n       Fib(n)   recursive calls   loop passes
  5            5               15             4
 10           55              177             9
 20         6765            21891            19
 30       832040          2692537            29
 35      9227465         29860703            34
```

For $n = 35$ the recursive version makes almost 30 million calls ($2 \cdot 14\,930\,352 - 1$), the loop 34 passes. Recursion is elegant, but a recursion that recomputes the same values can be exponentially

slow.

Stack overflow: recursion without a base case

If the base case is missing (`return n * Factorial(n - 1);` only), every call makes another call: $5, 4, \ldots, 0, -1, -2, \ldots$ The stack (about 1 MB) fills up and the process is killed. The compiler gives *no* warning. The real run ends with

```
dotnet run – process aborted, exit code 134

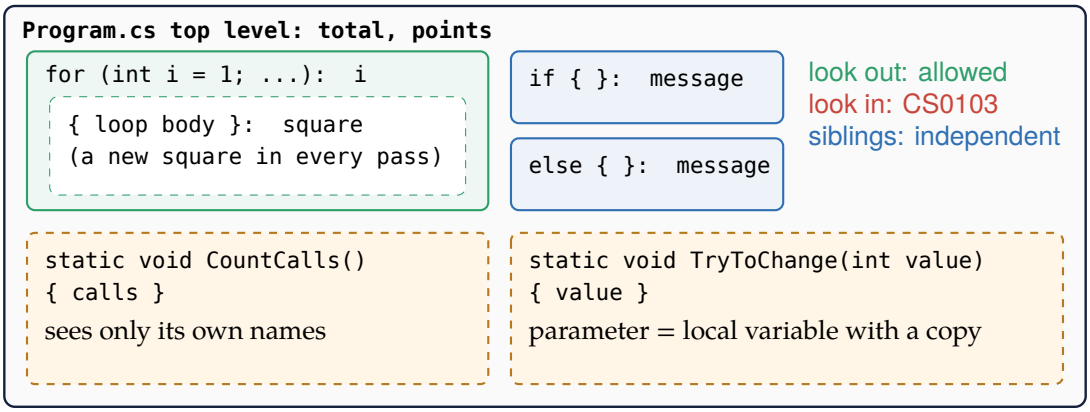
Stack overflow.
Repeated 174131 times:
------------------------------
   at Program.<<Main>$>g__Factorial|0_0(Int32)
```

(the repeat count depends on the machine). Checklist for every recursive method: is there a base case? does every call move towards it? is the depth reasonable (thousands, not millions)? Otherwise use a loop.

>_ 7 Scope and lifetime

7.1 Block scope

The **scope** of a variable is the part of the program text where its name can be used. A local variable's scope starts at its declaration and ends at the closing brace `}` of the block in which it is declared. Picture every block as a box:



- **Look out — allowed.** An inner block sees the names of every enclosing block: the loop body may use `total`.
- **Look in — not allowed.** After the loop, `square` and `i` do not exist: `CS0103 The name 'square' does not exist in the current context`. If you need a value after a block, declare the variable *before* the block.
- **Siblings are independent.** The `if` block and the `else` block may both declare a variable `message`; two consecutive `for` loops may both use `i`.
- **No reuse in nested blocks.** A nested block may not declare a name that is already used in an enclosing block: `CS0136`. Two declarations in the same block give `CS0128`. (C and C++ allow such "shadowing"; C# forbids it because it causes bugs.)

7.2 Method scope and lifetime

Parameters and local variables of a method belong to the method's own box. Their **lifetime** — the time they exist in memory — is one call: they are created when the method starts and destroyed when it returns. A local variable therefore remembers nothing between calls (`CountCalls` prints 1 three times), and changing a value parameter changes only the method's copy (`TryToChange`). To keep a value longer, return it to the caller, or let the caller pass a `ref` variable (as in Example 7).

7.3 Example 8 – scope and lifetime, with the real compiler errors

The commented-out lines do not compile. Each was uncommented on its own and built with `dotnet build`; the comment shows the real message.

Listing 8: 13_08_BlockScope/Program.cs

```
// MCMP113 · Week 13 · Example 8 — scope and lifetime of variables
// A variable lives from its declaration to the closing } of its block.
// The commented-out lines do NOT compile; the real compiler error is written next to them.

int total = 0;                          // scope: the rest of the top-level code
for (int i = 1; i <= 3; i++)            // i: only inside the for statement
{
    int square = i * i;                 // square: only inside this { }, new each pass
    total += square;
}
// Console.WriteLine(square);  // CS0103: The name 'square' does not exist in the current context
// Console.WriteLine(i);       // CS0103: The name 'i' does not exist in the current context
Console.WriteLine($"total = {total}");

if (total > 10)
{
    string message = "big";             // two SIBLING blocks may reuse a name
    Console.WriteLine(message);
}
else
{
    string message = "small";
    Console.WriteLine(message);
}

for (int k = 0; k < 2; k++)
{
    // int total = k;          // CS0136: A local or parameter named 'total' cannot be declared
                               //         in this scope because that name is used in an
                               //         enclosing local scope to define a local or parameter
    Console.WriteLine($"k = {k}, total = {total}");
}
// int total = 5;              // CS0128: A local variable or function named 'total' is already
                               //         defined in this scope

// Lifetime: a local variable is created again at every call
CountCalls();
CountCalls();
CountCalls();

// A parameter is a local variable that holds a COPY of the argument
int points = 40;
TryToChange(points);
Console.WriteLine($"after TryToChange: points = {points}");

// Prints how often it has been called... or does it?
static void CountCalls()
{
```

```
    int calls = 0;                      // born here, dies at the closing }
    calls++;
    Console.WriteLine($"CountCalls: calls = {calls}");
}

// Changes only its own copy of the argument
static void TryToChange(int value)
{
    value = 100;
    Console.WriteLine($"inside TryToChange: value = {value}");
}
```

Sample run

```
total = 14
big
k = 0, total = 14
k = 1, total = 14
CountCalls: calls = 1
CountCalls: calls = 1
CountCalls: calls = 1
inside TryToChange: value = 100
after TryToChange: points = 40
```

7.4 Local functions: capturing versus static

In a top-level program, methods are **local functions** of the top-level code. A *non-static* local function can read and change the top-level variables — it **captures** them. `AddBonus` in Example 9 uses `bonus`; when `bonus` changes somewhere else, the same call `AddBonus(70)` returns a different result. Such a top-level variable behaves like a **global variable**:

- it is a hidden input: the method header does not show it;
- the order of declarations matters: calling `AddBonus` before the line `int bonus = 5;` gives `CS0165 Use of unassigned local variable 'bonus';`
- results depend on history, so the method is hard to test.

A `static` local function sees only its parameters, its own locals and constants; using a top-level variable is a compile error, `CS8421 A static local function cannot contain a reference to 'bonus'.` Constants (`const double VatRate = 0.20;`) are fine everywhere, because they never change. **Course rule:** write local functions as `static`, pass in what they need, give back what they compute.

One exception to "no reuse of names": since C# 8 a parameter or local of a local *function* may have the same name as a top-level variable; it hides that variable inside the function (`Square(int n)` in Example 9). It compiles, but it is confusing — choose different names.

7.5 Example 9 – local functions

Listing 9: 13_09_LocalFunctions/Program.cs

```
// MCMP113 · Week 13 · Example 9 — local functions: capturing variables vs static
// A non-static local function can see the top-level variables; a static one cannot.

int bonus = 5;                          // a top-level variable ("global" for local functions)
Console.WriteLine($"AddBonus(70) = {AddBonus(70)}");
bonus = 10;                             // change it somewhere far away ...
Console.WriteLine($"AddBonus(70) = {AddBonus(70)}   (same call, new result!)");
Console.WriteLine($"AddPoints(70, 5) = {AddPoints(70, 5)}   (everything it needs is a parameter)");
```

```
9
10 // Hidden state: Visit() changes a variable that is not in its parameter list
11 int visits = 0;
12 Visit();
13 Visit();
14 Console.WriteLine($"visits = {visits}");
15
16 // Shadowing: the parameter n of Square hides the top-level n (allowed, but confusing)
17 int n = 4;
18 Console.WriteLine($"Square(9) = {Square(9)}, top-level n is still {n}");
19
20 // Captures 'bonus': its result depends on a variable outside the method
21 int AddBonus(int grade)
22 {
23     return grade + bonus;
24 }
25
26 // Static: can use only its parameters and constants
27 static int AddPoints(int grade, int points)
28 {
29     // return grade + bonus;  // CS8421: A static local function cannot contain a reference to '
       bonus'.
30     return grade + points;
31 }
32
33 // Side effect on a captured variable
34 void Visit()
35 {
36     visits++;
37 }
38
39 // Returns n * n for ITS OWN parameter n
40 static int Square(int n)
41 {
42     return n * n;
43 }
```

Sample run

```
AddBonus(70) = 75
AddBonus(70) = 80   (same call, new result!)
AddPoints(70, 5) = 75   (everything it needs is a parameter)
visits = 2
Square(9) = 81, top-level n is still 4
```

>_ 8 Testing methods and small utility libraries

Pure methods can be tested automatically: call them with inputs whose answers you know, compare, and print `PASS` or `FAIL`. Choose *boundary values* (Week 5): `LetterGrade(90)` must be AA, `LetterGrade(89.5)` must be BA, `LetterGrade(59.9)` must be FF. Run the tests after every change — a test that used to pass and now fails tells you that the last change broke something. Professional projects use test frameworks (xUnit, NUnit, MSTest) built on the same idea: `Assert.Equal(expected, actual)`.

8.1 Example 10 – a grade utility library with self-tests (excerpt)

The lines below are the self-tests and the test helper `Check`; lines 17–32 print the report of the sample run with the same methods (`Average`, `LetterGrade`, `GradePoint`, `IsPassing`). Their code (the Week 5 scale as a switch expression for `LetterGrade`) and `CountTrue` complete the file in the

code folder.

Listing 10: 13_10_GradeUtils/Program.cs (lines 1–15 and 83–90)

```
1  // MCMP113 · Week 13 · Example 10 — a small grade utility "library" with self-tests
2  // Each method does ONE job and returns a value; Check() prints PASS/FAIL for known answers.
3
4  Console.WriteLine("--- Self-tests ---");
5  bool[] results =
6  {
7      Check("Average {50, 70, 90} = 70", Average(new int[] { 50, 70, 90 }) == 70),
8      Check("LetterGrade(90) = AA", LetterGrade(90) == "AA"),
9      Check("LetterGrade(89.5) = BA", LetterGrade(89.5) == "BA"),
10     Check("LetterGrade(59.9) = FF", LetterGrade(59.9) == "FF"),
11     Check("GradePoint(\"CB\") = 2.5", GradePoint("CB") == 2.5),
12     Check("IsPassing(\"DD\")", IsPassing("DD")),
13     Check("!IsPassing(\"FF\")", !IsPassing("FF")),
14 };
15 Console.WriteLine($"{CountTrue(results)} of {results.Length} tests passed");
```

... lines 16–82 omitted ...

```
83 // Prints PASS or FAIL with a description and returns the result
84 static bool Check(string description, bool ok)
85 {
86     string verdict = ok ? "PASS" : "FAIL";
87     Console.WriteLine($"{verdict}  {description}");
88     return ok;
89 }
```

Sample run

```
--- Self-tests ---
PASS  Average {50, 70, 90} = 70
PASS  LetterGrade(90) = AA
PASS  LetterGrade(89.5) = BA
PASS  LetterGrade(59.9) = FF
PASS  GradePoint("CB") = 2.5
PASS  IsPassing("DD")
PASS  !IsPassing("FF")
7 of 7 tests passed
--- Report ---
Ayse      85.00  BA  3.5  passed
Mehmet    55.00  FF  0.0  failed
Zeynep    91.33  AA  4.0  passed
Can       50.00  FF  0.0  failed
```

8.2 Example 11 – prime utilities (excerpt)

`IsPrime` (trial division up to $\sqrt{n}$, Week 6) is written once and reused by `NextPrime`, `CountPrimes` and `PrimesUpTo`; `PrimesUpTo` even calls `CountPrimes` to know the length of the array it returns. A bug in `IsPrime` would be fixed in one place.

Listing 11: 13_11_PrimeUtils/Program.cs (lines 1–32)

```
1  // MCMP113 · Week 13 · Example 11 — prime utilities built from one predicate
2  // IsPrime is written once and reused by NextPrime, CountPrimes and PrimesUpTo.
3
4  Console.Write("Upper limit n: ");
5  int n = int.Parse(Console.ReadLine());
6
7  int[] primes = PrimesUpTo(n);
8  Console.WriteLine($"Primes up to {n}: {string.Join(" ", primes)}");
9  Console.WriteLine($"Count: {CountPrimes(n)}, next prime after {n}: {NextPrime(n)}");
10
```

```
11 Console.WriteLine("--- Self-tests ---");
12 Console.WriteLine(Check(!IsPrime(1) && IsPrime(2) && IsPrime(97) && !IsPrime(91), "IsPrime 1, 2, 97,
       91"));
13 Console.WriteLine(Check(NextPrime(13) == 17 && NextPrime(1) == 2, "NextPrime(13) = 17, NextPrime(1)
       = 2"));
14 Console.WriteLine(Check(CountPrimes(100) == 25, "CountPrimes(100) = 25"));
15
16 // True if n has exactly two divisors; trial division up to the square root
17 static bool IsPrime(int n)
18 {
19     if (n < 2)
20     {
21         return false;
22     }
23     for (int d = 2; d * d <= n; d++)
24     {
25         if (n % d == 0)
26         {
27             return false;               // a divisor found: stop at once
28         }
29     }
30     return true;
31 }
```

Sample run

```
Upper limit n: 50
Primes up to 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
Count: 15, next prime after 50: 53
--- Self-tests ---
PASS  IsPrime 1, 2, 97, 91
PASS  NextPrime(13) = 17, NextPrime(1) = 2
PASS  CountPrimes(100) = 25
```

8.3 Example 12 – string helpers and a conversion pair (excerpt)

`IsPalindrome` reuses `Reverse`; the temperature pair is checked with a *round trip* (there and back must give the start value, compared with a tolerance because of rounding, Week 4).

Listing 12: 13_12_StringHelpers/Program.cs (lines 1–13 and 49–55)

```
1  // MCMP113 · Week 13 · Example 12 — string helpers and a conversion pair, with self-tests
2  // Small pure methods: CountVowels, Reverse, IsPalindrome, CelsiusToFahrenheit and back.
3
4  Console.Write("Enter a word or sentence: ");
5  string text = Console.ReadLine();
6  Console.WriteLine($"Vowels: {CountVowels(text)}");
7  Console.WriteLine($"Reversed: {Reverse(text)}");
8  Console.WriteLine($"Palindrome? {IsPalindrome(text)}");
9
10 double c = 36.6;
11 double f = CelsiusToFahrenheit(c);
12 Console.WriteLine($"{c} C = {f:F2} F, and back: {FahrenheitToCelsius(f):F2} C");
```

... lines 14–48 omitted ...

```
49 // True if s reads the same backwards, ignoring spaces and upper/lower case
50 static bool IsPalindrome(string s)
51 {
52     string letters = s.Replace(" ", "").ToLower();
53     return letters == Reverse(letters);    // reuse Reverse
54 }
```

Lines 14–22 run six self-tests with a helper `Test` that prints PASS/FAIL and returns 1 for a failure, so

the failures can be added up; the round trip is `Math.Abs(FahrenheitToCelsius(CelsiusToFahrenheit(-40.5)) + 40.5) < 1e-9`.

Sample run 1

```
Enter a word or sentence: Kayak
Vowels: 2
Reversed: kayaK
Palindrome? True
36.6 C = 97.88 F, and back: 36.60 C
```

Sample run 2

```
Enter a word or sentence: Okan University
Vowels: 6
Reversed: ytisrevinU nakO
Palindrome? False
36.6 C = 97.88 F, and back: 36.60 C
```

Both runs then print the same six `PASS` lines and `All tests passed.`

>_ 9 Common mistakes

Return and scope mistakes

Mistake	Effect and fix
a path without `return`	`CS0161`; end every path with `return`.
printing inside a computing method	the result cannot be reused or tested; return it, print in the caller.
`Square(5);` alone	the result is thrown away; store or use it: `int sq = Square(5);`
`int y = PrintTitle("A");`	`CS0029`: a `void` call has no value.
`return n % 2 == 1;` for "odd"	wrong for negative n; use `n % 2 != 0`.
using a variable after its block	`CS0103`; declare it before the block.
the same name in a nested block	`CS0136`; choose another name.
recursion without base case	stack overflow at run time, no compiler warning.
a method changes its array parameter	the caller's data change; work on a copy or say so in the name.
a static local function uses a top-level variable	`CS8421`; pass it as a parameter.

>_ 10 Summary

- `return expr;` hands back one value and ends the method. Every path of a value-returning method must return (CS0161). A `void` method may leave early with `return;`.
- `void` methods perform actions; value-returning methods compute. Return results instead of printing them inside.
- `bool` predicates (`IsEven`, `IsPrime`, `IsValidTCKimlik`) return a condition; name them as questions.
- A method can return an array; two or more results travel in `out` parameters or in a tuple `(int min, int max)`.
- Calls compose from the inside out. Pure functions depend only on their arguments and change nothing else.
- Recursion needs a base case and a step towards it; each call has its own stack frame. Naive recursive Fibonacci makes $2F_{n+1} - 1$ calls.

- A local variable lives in its block (scope) and during one call (lifetime). Inner blocks see outer names, not the other way round (CS0103); nested blocks may not reuse names (CS0136).
- Prefer `static` local functions; share constants, not variables. Check methods with known inputs and PASS/FAIL output.

>_ 11 Glossary (English – Turkish)

English	Türkçe	English	Türkçe
return value	dönüş değeri	recursion	özyineleme (rekürsiyon)
return type	dönüş tipi	base case	temel durum
void method	değer döndürmeyen metot	recursive step	özyinelemeli adım
value-returning method	değer döndüren metot	call stack	çağrı yığını
early return	erken dönüş	stack frame	yığın çerçevesi
guard clause	koruma koşulu	stack overflow	yığın taşması
predicate	yüklem (bool döndüren metot)	scope	kapsam (geçerlilik alanı)
out parameter	çıkış (out) parametresi	lifetime	yaşam süresi
tuple	demet (tuple)	shadowing	gölgeleme
composition	iç içe çağırma (bileşim)	local function	yerel fonksiyon
pure function	saf fonksiyon	capture	yakalama
side effect	yan etki	global variable	genel (global) değişken
check digit	kontrol basamağı	unit test	birim testi

>_ 12 Exercises

12.1 Check yourself: predict the output

Write the exact output of Q1–Q3 and find the errors in Q4 (with the error codes). Then run the code to check.

Q1

```
int x = 5;
Change(x);
Console.WriteLine(x);
Console.WriteLine(Twice(Twice(x)));

static void Change(int x)
{
    x = x * 10;
}

static int Twice(int v)
{
    return 2 * v;
}
```

Q2

```
Show(3);

static void Show(int n)
{
    if (n == 0)
    {
        return;
    }
    Console.Write($"{n} ");
    Show(n - 1);
    Console.Write($"{n} ");
}
```

Q3

```
Console.WriteLine(Test(7));
Console.WriteLine(Test(-3));

static string Test(int n)
{
    if (n > 5)
    {
        return "big";
    }
    Console.WriteLine("checking...");
    if (n < 0)
    {
        return "negative";
    }
    return "small";
}
```

Q4 – which lines do not compile?

```
1  int sum = 0;
2  for (int i = 1; i <= 4; i++)
3  {
4      int sum = i * i;
5      sum += i;
6  }
7  if (sum > 5)
8  {
9      string label = "large";
10 }
11 Console.WriteLine($"{sum} {label} {i}");
```

12.2 In-class exercises (pairs)

E1. ⋆ **IsLeapYear with tests.** Write `static bool IsLeapYear(int year)` (divisible by 4, except centuries that are not divisible by 400) that returns the condition directly, and four PASS/FAIL checks for 2024, 1900, 2000 and 2026.

E2. ⋆⋆ **Recursive power.** Write `static double Power(double b, int e)` for $e \geq 0$ with recursion ($b^0 = 1$, $b^e = b \cdot b^{e-1}$). Trace `Power(2, 5)`.

E3. ⋆⋆ **Three results.** Write `Stats(int[] a)` that returns `(int min, int max, double mean)` as one tuple. Test it with `{4, 8, 15, 16, 23, 42}`.

E4. ⋆ **Scope detective.** Mark every line of the following program that does not compile, give the error code and fix it.

```
1  int count = 0;
2  for (int k = 0; k < 3; k++)
3  {
4      int doubled = k * 2;
5      count += doubled;
6  }
7  Console.WriteLine(doubled);
8  if (count > 3)
9  {
10     int count = 1;
11 }
12 string note = "done";
13 string note = "again";
14 Console.WriteLine(Half(count));
15
16 static int Half(int n)
17 {
18     if (n > 0)
19     {
20         return n / 2;
21     }
22 }
```

12.3 Homework

H1. ⋆⋆ **Check-digit generator.** Write `static string CompleteTCKimlik(string first9)` that returns the nine digits followed by d_{10} and d_{11}. Test: `123456789` → `12345678950`. Then check the result with `IsValidTCKimlik` from Example 2. Use invented numbers only.

H2. ⋆⋆ **ReadIntInRange.** Write `static int ReadIntInRange(string prompt, int min, int max)` that asks again until `int.TryParse` succeeds and the value lies in $[min, max]$, and returns the value.

H3. ⋆⋆⋆ **Recursive binary search.** Write `static int Find(int[] a, int target, int low, int high)` that returns the index of `target` in the sorted range `a[low..high]` or -1. Test it with

the array of Week 10, Example 5.

H4. ⋆ **Conversion table.** Using `CelsiusToFahrenheit`, print a table for $-10, 0, \ldots, 40\,^{\circ}\mathrm{C}$.

12.4 Extra practice

P1. ⋆ Trace `Gcd(252, 105)` (Example 6): list every call and the result.

P2. ⋆ How many calls does the naive `FibRec` make for $n = 6$? For $n = 10$?

P3. ⋆ Pure or not? (a) `Square`; (b) `PrintTitle`; (c) `CurveAndAverage`; (d) a method that returns `Random.Shared.Next(1, 7)`; (e) `AddBonus` of Example 9.

P4. ⋆⋆ What does `Mystery(5)` return? What does `Mystery(n)` compute in general?

```
Console.WriteLine(Mystery(5));

static int Mystery(int n)
{
    if (n == 0)
    {
        return 0;
    }
    return 2 * n - 1 + Mystery(n - 1);
}
```

P5. ⋆⋆ Write a recursive `static int CountDigits(int n)` for $n \geq 0$ (a one-digit number has one digit).

>_ 13 References

- Microsoft Learn — Methods in C#: https://learn.microsoft.com/en-us/dotnet/csharp/methods
- Microsoft Learn — Jump statements (`return`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/jump-statements
- Microsoft Learn — Local functions (including `static` local functions): https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/local-functions
- Microsoft Learn — Top-level statements: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/program-structure/top-level-statements
- Microsoft Learn — Tuple types: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/value-tuples
- Microsoft Learn — Method parameters (`out`, `ref`): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/method-parameters
- Microsoft Learn — Compiler errors CS0161, CS0136, CS0103: https://learn.microsoft.com/en-us/dotnet/csharp/misc/cs0161 https://learn.microsoft.com/en-us/dotnet/csharp/misc/cs0136 https://learn.microsoft.com/en-us/dotnet/csharp/misc/cs0103
- Microsoft Learn — Unit testing C# with xUnit: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit